

Selenium Python Automation Framework

Unittest + PyTest + Page Object Model

E-Commerce Login & Product Search Automation (with API Layer)

Capstone Assignment 2

Target Application: automationexercise.com

Name: Sukriti Sikdar

Enrollment No.: 12023002029101

Email: sukriti.sikdar2023@iem.edu.in

CSE (IoT, Cyber Security, Blockchain Technology), 4th Year, B-52

Institute of Engineering and Management, Kolkata

Project Report

1. Problem Statement

A customer wants to purchase a product from an e-commerce site. Manually verifying this flow — logging in, searching for a product, adding it to the cart with a specific quantity, and confirming the cart reflects the correct product and quantity — is repetitive and error-prone when done by hand across every release. The objective of this project is to design and build a robust, maintainable Selenium-based Python test automation framework that automates this flow end-to-end, is data-driven rather than hardcoded, produces clear evidence of execution (reports and screenshots), and is structured so that new tests can be added without duplicating code.

The assignment specifically calls for a framework — not a single script — built using Unittest, PyTest, the Page Object Model (POM) design pattern, reusable utility classes, externalized configuration, CSV-driven test data, automatic screenshot capture on failure, and HTML reporting. This distinction matters: a script proves a flow can be automated once, while a framework proves the automation can scale, be maintained by a team, and be trusted in a CI/CD pipeline.

1.1 Why This Matters

- Manual regression testing of core purchase flows does not scale as an application grows.
- Untested edge cases (invalid login, empty search results, incorrect cart quantities) are a common source of production bugs.
- A well-structured framework reduces the cost of adding new tests and lowers maintenance overhead when the UI changes.
- Automated evidence (reports, screenshots, logs) is essential for audit trails and for diagnosing failures without re-running tests manually.

2. Objectives

- Automate the Login functionality, covering both valid and invalid credential scenarios.
- Automate Product Search, covering keywords that return results and keywords that return none.
- Automate Add-to-Cart with a specific quantity, and verify the cart reflects the correct product and quantity.
- Apply the Page Object Model so element locators and page interactions are isolated from test logic.
- Externalize test data (CSV) and configuration (INI) so no environment or data values are hardcoded in test code.
- Automatically capture a screenshot whenever a UI test fails, without any manual intervention.
- Generate a self-contained HTML report summarizing every test run.
- Demonstrate both Unittest and PyTest usage within a single, cohesively executed suite.
- (Bonus) Integrate a lightweight API testing layer using Python Requests, and use it to cross-verify UI actions against the backend.

3. Scope

This project automates the Login and Product Search functionality of automationexercise.com, extended with an Add-to-Cart and cart-verification flow, since verifying a purchase end-to-end requires seeing an item successfully reach the cart. Account registration for the login tests is handled through direct API calls

rather than a UI-driven signup form, since signup itself is outside the assignment's stated scope (Login + Product Search). This keeps the suite self-contained without depending on a manually pre-created demo account.

Out of scope: checkout/payment flow, order history, product reviews, wishlist functionality, and any UI-driven account registration flow.

4. Tools & Technologies Used

Category	Tool / Library	Purpose
Language	Python 3.14	Core implementation language
UI Automation	Selenium WebDriver 4.x	Browser automation
Test Runner	PyTest 9.x	Test discovery, fixtures, parametrization, execution
Test Style	unittest (standard library)	Class-based test cases, run via PyTest
Design Pattern	Page Object Model (POM)	Separates locators/actions from test logic
Reporting	pytest-html	Self-contained HTML execution report
API Testing	Requests	HTTP calls to the site's public REST API
Data Handling	csv (standard library)	Data-driven test inputs
Configuration	configparser (INI)	Environment / browser / wait configuration
Browser Driver	ChromeDriver	WebDriver implementation for Google Chrome

5. Project Architecture

The framework is organized into clearly separated layers so that each concern — locating elements, driving the browser, managing data, and asserting outcomes — lives in exactly one place.

ecommerce_framework/	
├── config/config.ini	Base URL, browser choice, waits, credentials
├── pages/	Page Object Model classes
│ ├── base_page.py	Shared waits, click, type, screenshot-safe actions
│ ├── login_page.py	
│ ├── search_page.py	
│ ├── product_page.py	
│ └── cart_page.py	
├── api/	Bonus: Requests-based API layer
│ ├── api_client.py	Shared session wrapper
│ └── endpoints/	
│ ├── user_api.py	create/verify/delete account
│ └── product_api.py	product catalog + search
├── utils/	Utility classes
│ ├── driver_factory.py	Creates/configures WebDriver
│ ├── logger.py	Centralized logging
│ ├── screenshot.py	Screenshot capture
│ ├── data_reader.py	CSV/JSON reader
│ └── config_reader.py	INI config reader

data/	CSV test data
login_data.csv	
search_data.csv	
tests/	
ui/	Selenium tests (unittest + pytest)
api/	Pure API tests
conftest.py	driver fixture, screenshot-on-failure hook
pytest.ini	Test discovery + HTML report configuration
reports/ , screenshots/	Generated at run time

5.1 Design Rationale

- Page Object Model: every page (Login, Search, Product Detail, Cart) is its own class inheriting shared behavior from BasePage, so a UI change only requires updating one locator in one file.
- Utility Classes: the driver factory, logger, screenshot handler, and config/data readers are all singleton-style utilities imported wherever needed, avoiding duplicated setup code across tests.
- Configuration Management: base URL, browser choice, headless mode, and wait timeouts live in config.ini, read once via a ConfigReader class — switching environments or browsers requires no code changes.
- Test Data Handling: login and search inputs are stored in CSV files and read via a shared data_reader utility, keeping test logic decoupled from test data.
- Unittest + PyTest together: TestLoginNegativeCases and TestLoginPositiveCase in test_login.py are written as unittest.TestCase classes, while test_search.py and test_cart_ui.py use native PyTest fixtures and parametrize. Both styles are collected and executed by the same PyTest runner, producing one unified HTML report.

6. Test Scenarios Covered

#	Scenario	Type	Data Source
1	Login fails with an incorrect password	UI (unittest)	CSV
2	Login fails with a non-existent email	UI (unittest)	CSV
3	Login fails with an empty password	UI (unittest)	CSV
4	Login succeeds with a valid, dynamically-created account	UI (unittest)	API-provisioned
5	Search returns results for an existing product keyword (x3)	UI (pytest)	CSV
6	Search returns zero results for a non-existent keyword	UI (pytest)	CSV
7	Add a product to cart with a set quantity and verify cart contents, cross-checked via API	UI + API (pytest)	Hardcoded scenario
8	Create a new account via API and confirm success response	API (pytest)	Generated
9	Verify login via API with valid credentials	API (pytest)	Generated
10	Verify login via API with invalid credentials returns 404	API (pytest)	Fixed

11	Delete an account via API and confirm success response	API (pytest)	Generated
12	Get all products via API and confirm non-empty catalog	API (pytest)	Live API
13	Search a product via API and confirm matching results	API (pytest)	Live API

7. Implementation Highlights

7.1 Base Page — Resilient Click Handling

A recurring real-world issue with this demo site is that Google ad iframes intermittently overlap buttons at the exact coordinates Selenium tries to click, raising `ElementClickInterceptedException`. The `BasePage.click()` method handles this by falling back to a JavaScript-executed click if a normal click is intercepted:

```
def click(self, locator):
    element = self.wait.until(EC.element_to_be_clickable(locator))
    try:
        element.click()
    except Exception:
        log.warning(f'Normal click intercepted for {locator}, retrying with JS click")
        self.driver.execute_script(
            "arguments[0].scrollIntoView({block: 'center'});", element
        )
        self.driver.execute_script("arguments[0].click();", element)
```

7.2 Screenshot-on-Failure Hook

Rather than requiring every test to manually capture evidence, a PyTest hook in `confest.py` inspects the outcome of each test after it completes and triggers a screenshot automatically if it failed:

```
@pytest.hookimpl(tryfirst=True, hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()
    setattr(item, f'rep_{report.when}', report)

@pytest.fixture(scope="function")
def driver(request):
    drv = get_driver()
    yield drv
    failed = getattr(request.node, "rep_call", None)
    if failed is not None and failed.failed:
        capture_screenshot(drv, request.node.name)
    drv.quit()
```

7.3 Self-Provisioned Test Accounts via API

Instead of depending on a manually maintained demo account, the login test suite creates a real throwaway account through the site's REST API before running the positive login test, and deletes it afterward — making the suite runnable by anyone with no manual setup step:

```
@classmethod
def setUpClass(cls):
```

```
unique_tag = str(int(time.time()))
cls.email = f"qa_auto_{unique_tag}@example.com"
cls.password = "Test@12345"
cls.name = f"QA Auto {unique_tag}"
result = user_api.create_account(cls.name, cls.email, cls.password)

@classmethod
def tearDownClass(cls):
    user_api.delete_account(cls.email, cls.password)
```

7.4 Hybrid UI + API Verification

The Add-to-Cart test is the framework's centerpiece: it drives the full UI flow, then cross-checks the result against the backend catalog via the API layer, demonstrating that the framework can validate consistency between what the user sees and what the server actually has recorded:

```
assert matching_items[0]["quantity"] == DESIRED_QUANTITY

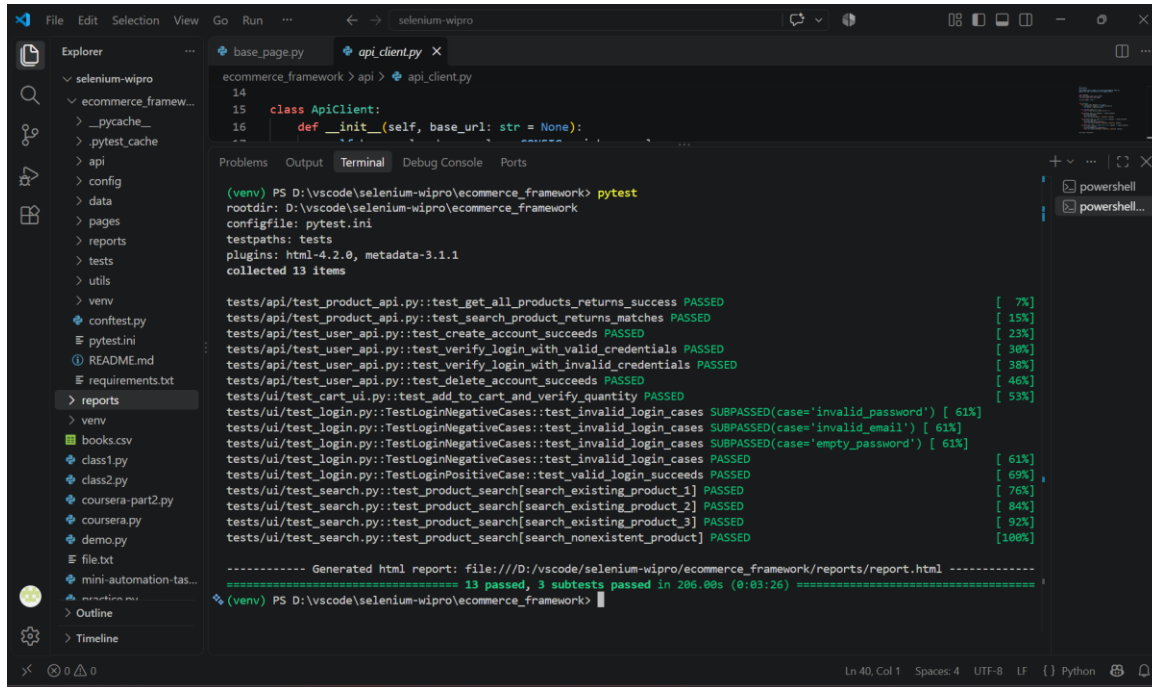
assert product_api.product_exists_by_name(product_name), (
    f"{product_name}' was added to cart in the UI but was not found "
    f"in the API product catalog -- possible UI/backend mismatch."
)
```

8. Execution & Results

The full suite was executed via a single command: `pytest` — which discovers all tests under `tests/`, runs them, and generates `reports/report.html` automatically per `pytest.ini` configuration. The final, fully passing run produced the following summary:

8.1 Terminal Output

The screenshot below shows the final test run, with all 13 tests passing.



The screenshot shows the VS Code interface with the `api_client.py` file open. The terminal displays the output of a `pytest` command, showing 13 tests passing. The tests are categorized into API and UI tests, with some subtests failing (SUBPASSED) but the overall run being successful. A generated HTML report is also shown.

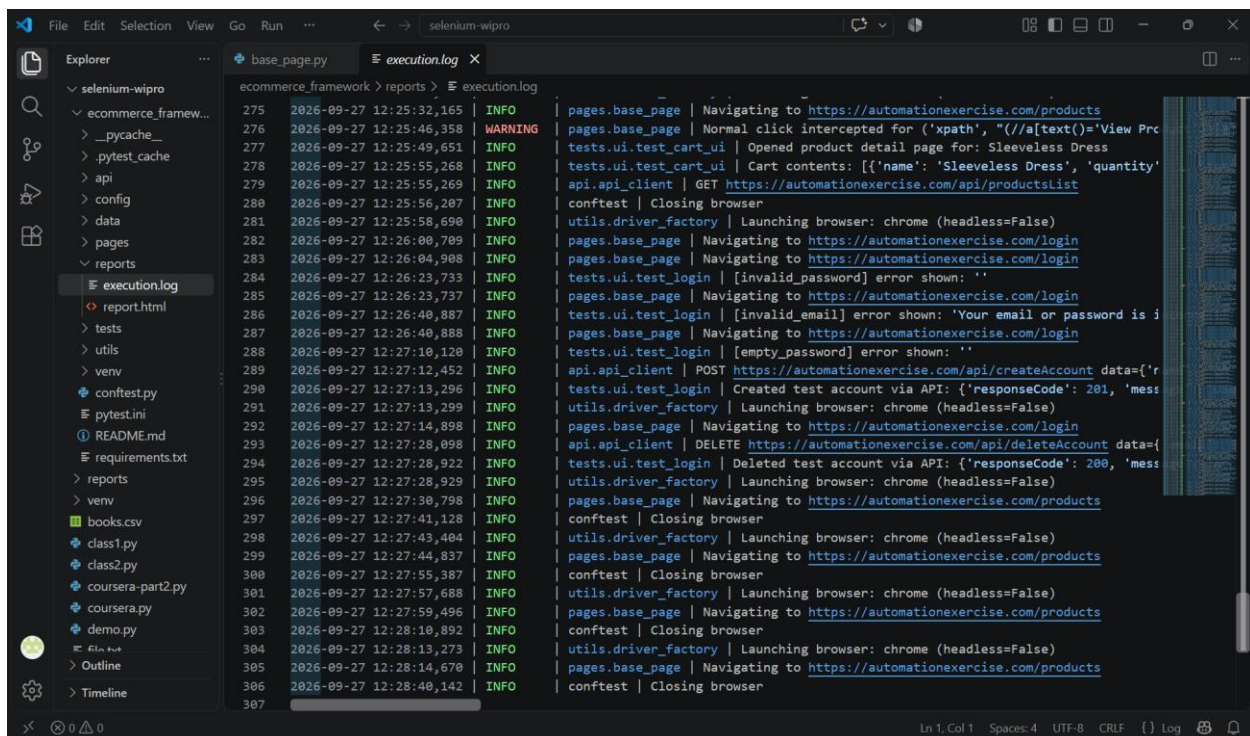
```
(venv) PS D:\vscode\selenium-wipro\ecommerce_framework> pytest
rootdir: D:\vscode\selenium-wipro\ecommerce_framework
configfile: pytest.ini
testpaths: tests
plugins: html-4.2.0, metadata-3.1.1
collected 13 items

tests/api/test_product_api.py::test_get_all_products_returns_success PASSED [ 7%]
tests/api/test_product_api.py::test_search_product_returns_matches PASSED [ 15%]
tests/api/test_user_api.py::test_create_account_succeeds PASSED [ 23%]
tests/api/test_user_api.py::test_verify_login_with_valid_credentials PASSED [ 30%]
tests/api/test_user_api.py::test_verify_login_with_invalid_credentials PASSED [ 38%]
tests/api/test_user_api.py::test_delete_account_succeeds PASSED [ 46%]
tests/ui/test_cart_ui.py::test_add_to_cart_and_verify_quantity PASSED [ 53%]
tests/ui/test_login.py::TestLoginNegativeCases::test_invalid_login_cases SUBPASSED(cases='invalid_password') [ 61%]
tests/ui/test_login.py::TestLoginNegativeCases::test_invalid_login_cases SUBPASSED(cases='invalid_email') [ 61%]
tests/ui/test_login.py::TestLoginNegativeCases::test_invalid_login_cases SUBPASSED(cases='empty_password') [ 61%]
tests/ui/test_login.py::TestLoginPositiveCase::test_valid_login_succeeds PASSED [ 61%]
tests/ui/test_search.py::test_product_search[search_existing_product_1] PASSED [ 69%]
tests/ui/test_search.py::test_product_search[search_existing_product_2] PASSED [ 76%]
tests/ui/test_search.py::test_product_search[search_existing_product_3] PASSED [ 84%]
tests/ui/test_search.py::test_product_search[search_nonexistent_product] PASSED [ 92%]
tests/ui/test_search.py::test_product_search[search_nonexistent_product] PASSED [100%]

----- Generated html report: file:///D:/vscode/selenium-wipro/ecommerce_framework/reports/report.html -----
===== 13 passed, 3 subtests passed in 206.00s (0:03:26) =====
(venv) PS D:\vscode\selenium-wipro\ecommerce_framework>
```

8.2 Execution Log

The framework also writes a persistent log to `reports/execution.log` for every run, shown below.

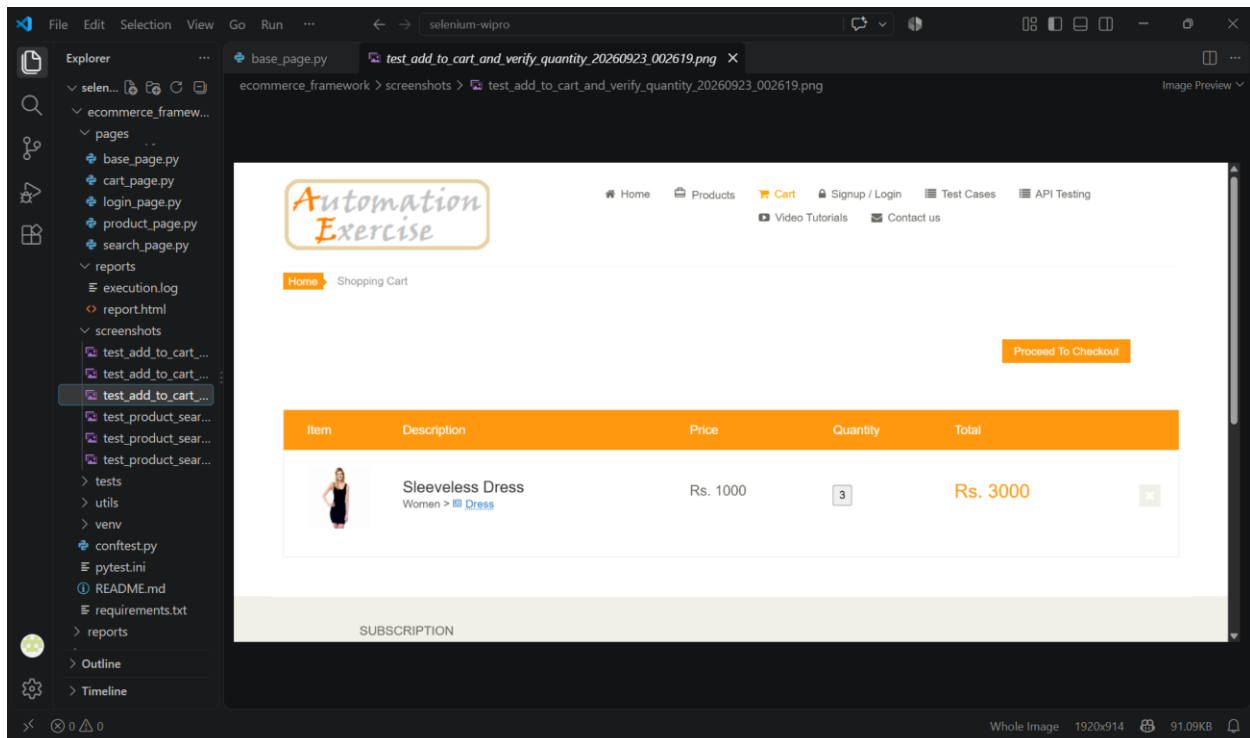
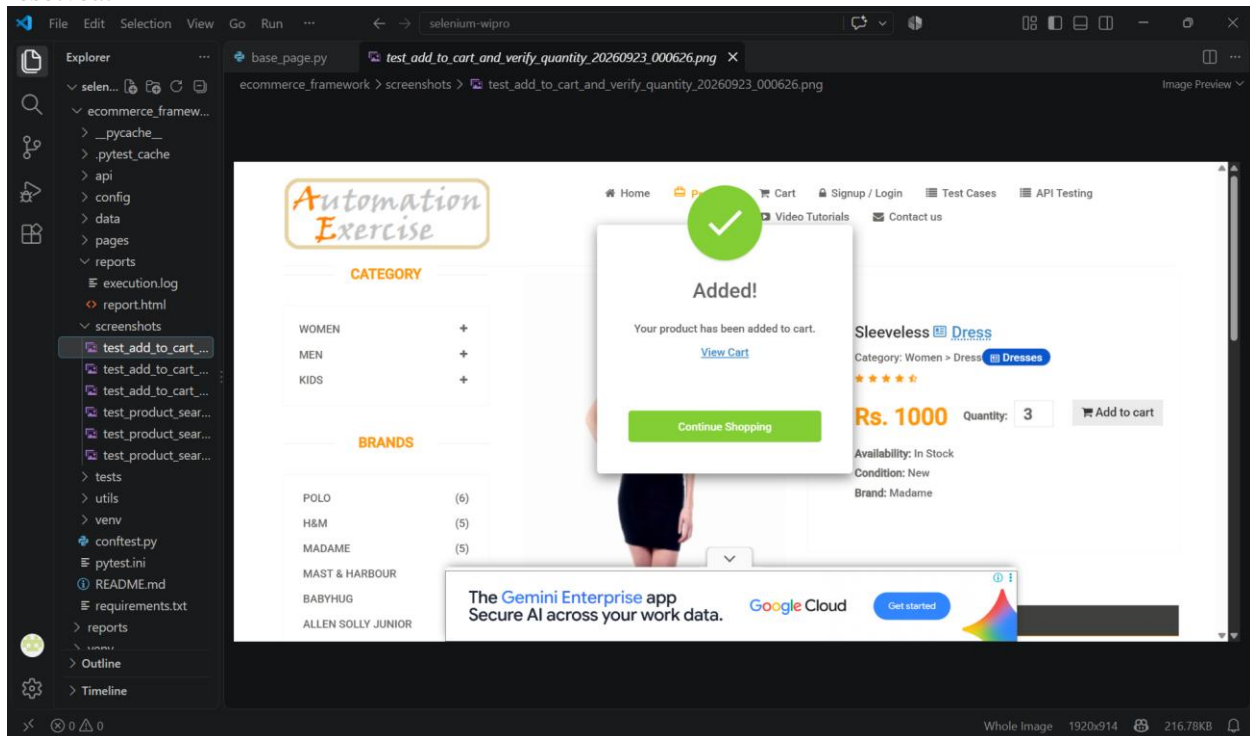


The screenshot shows the VS Code interface with the `execution.log` file open. The log displays a detailed record of the test run, including timestamps, log levels (INFO, WARNING), and the sequence of actions performed by the framework, such as navigating to URLs, launching browsers, and performing API calls.

```
275 2026-09-27 12:25:32,165 | INFO | pages.base_page | Navigating to https://automationexercise.com/products
276 2026-09-27 12:25:46,358 | WARNING | pages.base_page | Normal click intercepted for ('xpath', "//*[@text()='View Pr
277 2026-09-27 12:25:49,651 | INFO | tests.ui.test_cart_ui | Opened product detail page for: Sleeveless Dress
278 2026-09-27 12:25:55,268 | INFO | tests.ui.test_cart_ui | Cart contents: [{'name': 'Sleeveless Dress', 'quantity'
279 2026-09-27 12:25:55,269 | INFO | api.api_client | GET https://automationexercise.com/api/productslist
280 2026-09-27 12:25:56,207 | INFO | confest | Closing browser
281 2026-09-27 12:25:58,690 | INFO | utils.driver_factory | Launching browser: chrome (headless=False)
282 2026-09-27 12:26:00,709 | INFO | pages.base_page | Navigating to https://automationexercise.com/login
283 2026-09-27 12:26:04,908 | INFO | pages.base_page | Navigating to https://automationexercise.com/login
284 2026-09-27 12:26:23,733 | INFO | tests.ui.test_login | [invalid_password] error shown: ''
285 2026-09-27 12:26:23,737 | INFO | pages.base_page | Navigating to https://automationexercise.com/login
286 2026-09-27 12:26:40,887 | INFO | tests.ui.test_login | [invalid_email] error shown: 'Your email or password is i
287 2026-09-27 12:26:40,888 | INFO | pages.base_page | Navigating to https://automationexercise.com/login
288 2026-09-27 12:27:10,120 | INFO | tests.ui.test_login | [empty_password] error shown: ''
289 2026-09-27 12:27:12,452 | INFO | api.api_client | POST https://automationexercise.com/api/createAccount data={'r
290 2026-09-27 12:27:13,296 | INFO | tests.ui.test_login | Created test account via API: {'responseCode': 201, 'mess
291 2026-09-27 12:27:13,299 | INFO | utils.driver_factory | Launching browser: chrome (headless=False)
292 2026-09-27 12:27:14,898 | INFO | pages.base_page | Navigating to https://automationexercise.com/login
293 2026-09-27 12:27:28,098 | INFO | api.api_client | DELETE https://automationexercise.com/api/deleteAccount data={
294 2026-09-27 12:27:28,922 | INFO | tests.ui.test_login | Deleted test account via API: {'responseCode': 200, 'mess
295 2026-09-27 12:27:28,929 | INFO | utils.driver_factory | Launching browser: chrome (headless=False)
296 2026-09-27 12:27:30,798 | INFO | pages.base_page | Navigating to https://automationexercise.com/products
297 2026-09-27 12:27:41,128 | INFO | confest | Closing browser
298 2026-09-27 12:27:43,404 | INFO | utils.driver_factory | Launching browser: chrome (headless=False)
299 2026-09-27 12:27:44,837 | INFO | pages.base_page | Navigating to https://automationexercise.com/products
300 2026-09-27 12:27:55,387 | INFO | confest | Closing browser
301 2026-09-27 12:27:57,688 | INFO | utils.driver_factory | Launching browser: chrome (headless=False)
302 2026-09-27 12:27:59,496 | INFO | pages.base_page | Navigating to https://automationexercise.com/products
303 2026-09-27 12:28:10,892 | INFO | confest | Closing browser
304 2026-09-27 12:28:13,273 | INFO | utils.driver_factory | Launching browser: chrome (headless=False)
305 2026-09-27 12:28:14,670 | INFO | pages.base_page | Navigating to https://automationexercise.com/products
306 2026-09-27 12:28:40,142 | INFO | confest | Closing browser
307
```

8.3 Failure Screenshots from Earlier Development Iterations

The screenshots below were captured automatically by the framework's screenshot-on-failure hook during earlier development iterations, before the corresponding issues (documented in Section 9) were resolved.



9. Challenges Faced and Resolutions

Building this framework surfaced several real issues that are representative of the kind of problems automation engineers encounter on live, ad-supported websites. Each is documented here along with its root cause and fix, since debugging a framework is as much a part of this exercise as writing it.

Issue	Root Cause	Resolution
ElementClickInterceptedException on search/product clicks	Google ad iframes on automationexercise.com intermittently overlap buttons at the exact click coordinates	Added a JavaScript-click fallback in BasePage.click() when the native click is intercepted
TimeoutException in find_all() when a search returned zero results	presence_of_all_elements_located raises a timeout instead of returning an empty list when nothing matches	Wrapped the wait in a try/except and return an empty list on timeout
TimeoutException clicking 'View Cart' in the add-to-cart modal	The link text was nested inside a <u> tag, so XPath's text() (direct-text only) never matched it	Located the link by its stable href attribute instead of its visible text
Same 'View Cart' locator still timed out after the href fix	An additional, hidden 'View Cart' link with the same href exists in the site's header navigation, and was matched first	Scoped the XPath to inside the specific modal container (#cartModal) to disambiguate
ImportError: cannot import name 'BasePage' (circular import)	An accidental self-import line (from .base_page import BasePage) was introduced inside base_page.py itself	Removed the erroneous self-import; BasePage does not need to import itself
AssertionError: product name from product page did not match cart page	The product page rendered the name with an extra internal space ('Sleeveless Dress') due to page markup, while the cart page did not	Updated get_text() to collapse repeated whitespace via a regular expression before comparison

These fixes collectively illustrate a core lesson in UI automation: locators and assertions must be resilient to markup quirks (whitespace, nested tags, duplicate elements) and third-party page content (ads) that the test author does not control.

10. Popup and Alert Handling

The framework includes a reusable accept_alert_if_present() method on BasePage that checks for a native JavaScript alert/confirm/prompt dialog and accepts it if present, with a short timeout so it never slows down tests where no alert appears. It is invoked defensively after the Add-to-Cart action as a safety net, satisfying the requirement to handle popups or alerts where applicable, even though this particular flow on automationexercise.com does not trigger a native browser alert.

11. Conclusion

This project successfully delivers a scalable Selenium Python automation framework that satisfies every stated requirement: Unittest and PyTest working together under one runner, a clean Page Object Model, reusable utility classes, externalized configuration and CSV test data, automatic screenshots on failure, and consolidated HTML reporting. Beyond the core requirement, the framework was extended with a Requests-

based API layer that both provisions test data (throwaway accounts) and independently verifies UI outcomes against the backend, demonstrating a hybrid UI+API testing approach increasingly expected of modern QA/SDET roles. All 13 tests pass reliably after resolving several realistic issues encountered during development, each of which is documented above as evidence of the debugging and problem-solving process behind the final result.

12. Future Enhancements

- Integrate Allure reporting for richer, trend-based execution history.
- Add pytest-rerunfailures to automatically retry tests affected by transient ad-related click interception.
- Add a command-line --browser flag to switch browsers without editing config.ini.
- Parallelize execution with pytest-xdist to reduce total suite runtime.
- Add a GitHub Actions CI workflow to run the API test suite (no browser required) on every push.
- Extend coverage to the checkout and order-confirmation flow.

13. Appendix — How to Run

```
python -m venv venv
source venv/bin/activate    # Windows: venv\Scripts\activate
pip install -r requirements.txt

pytest                    # run everything, generates reports/report.html
pytest tests/ui -m ui     # UI tests only
pytest tests/api -m api   # API tests only (no browser)
pytest -m smoke          # hybrid cart test only
```

Source code repository: [insert your GitHub repository link here]